

探索性数据分析期末大作业

核心代码附录

花浩文 2462404009

苏州大学未来科学与工程学院

题目：纵向历史特征如何改善糖尿病患者 30 天再入院预测
2026 年 6 月

说明

本附录只保留论文方法与结论最核心的代码片段，便于纸质审阅。完整代码、最终论文、关键结果表和复现说明已上传至 GitHub: <https://github.com/HANA225-HUB/diabetes-readmission-eda-final>。

筛选原则如下：

1. 保留能体现数据审计、特征工程、患者级划分、指标选择和最终模型构造的代码；
2. 保留严格纵向历史特征、target-history 敏感性上限、去 gap 消融和患者级交叉验证；
3. 不在纸质版重复打印探索性搜索、绘图和临时验证脚本，避免附录过厚影响阅读。

片段	对应论文内容
数据准备	样本排除、目标变量、诊断/用药/医疗强度特征、数据审计。
患者级划分与指标	patient_nbr 互斥划分，AP/PR-AUC、Brier、F2 阈值和混淆矩阵指标。
纵向历史特征	前次诊断路径、前次出院去向、医疗强度变化、此前观测住院次数；target-history 作为敏感性上限。
最终历史模型	CatBoost 历史模型、验证集权重搜索、strict-history 与 sensitivity 分支。
稳健性验证	去除 encounter_id gap 的消融实验和五折患者级交叉验证。

1 核心代码

1.1 数据清洗与特征工程

来源: diabetes_readmission_project.py:190-276

作用: 读取课程数据, 排除死亡/临终关怀与无效性别记录, 构造 30 天再入院二分类目标, 并生成诊断、用药、检查、医疗强度和既往利用特征。

```
1 def prepare_data() -> tuple[pd.DataFrame, dict]:
2     raw = pd.read_csv(DATA_DIR / "diabetic_data.csv")
3     original_rows = len(raw)
4     question_missing = (raw.astype("string") == "?").sum().sort_values(ascending=False)
5     actual_missing = raw.isna().sum()
6     missing = pd.DataFrame({
7         "question_mark_missing": question_missing,
8         "actual_na": actual_missing,
9     })
10    missing["missing_total"] = missing.sum(axis=1)
11    missing["missing_rate"] = missing["missing_total"] / len(raw)
12    missing = missing.sort_values("missing_rate", ascending=False)
13    missing.to_csv(TAB / "01_missingness.csv", encoding="utf-8-sig")
14
15    death_hospice_codes = {11, 13, 14, 19, 20, 21}
16    excluded_death_hospice = raw["discharge_disposition_id"].isin(death_hospice_codes).sum()
17    excluded_invalid_gender = (raw["gender"] == "Unknown/Invalid").sum()
18    df = raw.loc[
19        ~raw["discharge_disposition_id"].isin(death_hospice_codes)
20        & (raw["gender"] != "Unknown/Invalid")
21    ].copy()
22
23    df["target_30d"] = (df["readmitted"] == "<30").astype(int)
24    df["admission_type_raw"] = df["admission_type_id"].astype(str)
25    df["discharge_disposition_raw"] = df["discharge_disposition_id"].astype(str)
26    df["admission_source_raw"] = df["admission_source_id"].astype(str)
27    for col in ["race", "gender", "age", "max_glu_serum", "A1Cresult", "change", "diabetesMed"] + MEDICATION_COLUMNS:
28        df[col] = clean_string(df[col])
29
30    df["age_mid"] = df["age"].map(age_midpoint)
31    df["race_clean"] = clean_string(df["race"])
32    df["medical_specialty_group"] = top_or_other(df["medical_specialty"], 12)
33    df["payer_available"] = np.where(df["payer_code"].astype("string").eq("?"), "Missing", "Recorded")
34    df["admission_type_group"] = df["admission_type_id"].map(admission_type_group)
35    df["admission_source_group"] = df["admission_source_id"].map(admission_source_group)
36    df["discharge_group"] = df["discharge_disposition_id"].map(discharge_group)
37
38    for idx in (1, 2, 3):
39        df[f"diag{idx}_group"] = df[f"diag_{idx}"].map(diagnosis_group)
40    df["primary_diabetes_dx"] = (df["diag1_group"] == "Diabetes").astype(int)
41    df["diagnosis_diversity"] = df[["diag1_group", "diag2_group", "diag3_group"]].nunique(axis=1)
42
43    medication = df[MEDICATION_COLUMNS]
44    df["active_med_count"] = medication.ne("No").sum(axis=1)
45    df["dose_change_count"] = medication.isin(["Up", "Down"]).sum(axis=1)
46    df["insulin_active"] = df["insulin"].ne("No").astype(int)
47    df["medication_changed"] = df["change"].eq("Ch").astype(int)
48    df["diabetes_med_active"] = df["diabetesMed"].eq("Yes").astype(int)
49
50    df["prior_visits_total"] = df[["number_outpatient", "number_emergency", "number_inpatient"]].sum(axis=1)
51    df["prior_inpatient_flag"] = (df["number_inpatient"] > 0).astype(int)
52    df["prior_emergency_flag"] = (df["number_emergency"] > 0).astype(int)
53    df["prior_outpatient_flag"] = (df["number_outpatient"] > 0).astype(int)
54    days = df["time_in_hospital"].clip(lower=1)
55    df["labs_per_day"] = df["num_lab_procedures"] / days
56    df["medications_per_day"] = df["num_medications"] / days
57    df["procedures_per_day"] = df["num_procedures"] / days
58    df["diagnoses_per_day"] = df["number_diagnoses"] / days
59    df["care_intensity"] = (
60        df["num_lab_procedures"] + 2 * df["num_procedures"] + df["num_medications"]
61    ) / days
62
63    df["A1C_tested"] = df["A1Cresult"].ne("Missing").astype(int)
64    df["A1C_abnormal"] = df["A1Cresult"].isin([>7, >8]).astype(int)
65    df["glucose_tested"] = df["max_glu_serum"].ne("Missing").astype(int)
66    df["glucose_abnormal"] = df["max_glu_serum"].isin([>200, >300]).astype(int)
67
68    repeat_counts = df.groupby("patient_nbr").size()
69    df["patient_encounter_count"] = df["patient_nbr"].map(repeat_counts)
70
71    audit = {
72        "original_rows": int(original_rows),
73        "columns": int(raw.shape[1]),
74        "excluded_death_hospice": int(excluded_death_hospice),
75        "excluded_invalid_gender": int(excluded_invalid_gender),
76        "analysis_rows": int(len(df)),
77        "unique_patients": int(df["patient_nbr"].nunique()),
78        "positive_count": int(df["target_30d"].sum()),
79        "positive_rate": float(df["target_30d"].mean()),
80        "repeat_patient_rate": float((repeat_counts > 1).mean()),
```

```
81     "max_encounters_per_patient": int(repeat_counts.max()),
82     "top_missing": {
83         k: float(v) for k, v in missing["missing_rate"].head(8).items()
84     },
85 }
86 (RES / "data_audit.json").write_text(json.dumps(audit, ensure_ascii=False, indent=2), encoding="utf-8")
87 return df, audit
```

1.2 患者级划分与评价指标

来源: diabetes_readmission_project.py:408-512

作用: 按患者编号而非住院记录划分训练、验证、测试集, 避免同一患者跨集合泄漏; 同时定义 AP/PR-AUC、Brier、F2 阈值和混淆矩阵指标。

```
1 def patient_level_split(df: pd.DataFrame) -> tuple[np.ndarray, np.ndarray, np.ndarray, dict]:
2     patient_target = df.groupby("patient_nbr")["target_30d"].max()
3     patients = patient_target.index.to_numpy()
4     labels = patient_target.to_numpy()
5     train_patients, temp_patients = train_test_split(
6         patients, test_size=0.36, random_state=SEED, stratify=labels
7     )
8     temp_labels = patient_target.loc[temp_patients].to_numpy()
9     val_patients, test_patients = train_test_split(
10        temp_patients, test_size=20 / 36, random_state=SEED, stratify=temp_labels
11    )
12    train_idx = df.index[df["patient_nbr"].isin(train_patients)].to_numpy()
13    val_idx = df.index[df["patient_nbr"].isin(val_patients)].to_numpy()
14    test_idx = df.index[df["patient_nbr"].isin(test_patients)].to_numpy()
15    split = {
16        "train_rows": int(len(train_idx)), "validation_rows": int(len(val_idx)), "test_rows": int(len(test_idx)),
17        "train_patients": int(len(train_patients)), "validation_patients": int(len(val_patients)),
18        "test_patients": int(len(test_patients)),
19        "train_rate": float(df.loc[train_idx, "target_30d"].mean()),
20        "validation_rate": float(df.loc[val_idx, "target_30d"].mean()),
21        "test_rate": float(df.loc[test_idx, "target_30d"].mean()),
22    }
23    (RES / "split_summary.json").write_text(json.dumps(split, ensure_ascii=False, indent=2), encoding="utf-8")
24    return train_idx, val_idx, test_idx, split
25
26
27 def feature_sets() -> dict[str, tuple[list[str], list[str]]]:
28     cleaned_num = BASE_NUMERIC + ["age_mid"]
29     cleaned_cat = [
30         "race_clean", "gender", "admission_type_group", "admission_source_group", "discharge_group",
31         "medical_specialty_group", "payer_available", "max_glu_serum", "A1Cresult", "change", "diabetesMed",
32     ] + MEDICATION_COLUMNS
33     engineered_num = cleaned_num + [
34         "primary_diabetes_dx", "diagnosis_diversity", "active_med_count", "dose_change_count",
35         "insulin_active", "medication_changed", "diabetes_med_active", "prior_visits_total",
36         "prior_inpatient_flag", "prior_emergency_flag", "prior_outpatient_flag", "labs_per_day",
37         "medications_per_day", "procedures_per_day", "diagnoses_per_day", "care_intensity",
38         "A1C_tested", "A1C_abnormal", "glucose_tested", "glucose_abnormal",
39     ]
40     engineered_cat = cleaned_cat + ["diag1_group", "diag2_group", "diag3_group"]
41     return {
42         "Raw": (BASE_NUMERIC, BASE_CATEGORICAL),
43         "Cleaned": (cleaned_num, cleaned_cat),
44         "Engineered": (engineered_num, engineered_cat),
45     }
46
47
48 def make_pipeline(model_name: str, numeric: list[str], categorical: list[str]) -> Pipeline:
49     if model_name == "Logistic Regression":
50         pre = ColumnTransformer([
51             ("num", Pipeline([
52                 ("imputer", SimpleImputer(strategy="median")),
53                 ("scaler", StandardScaler()),
54             ]), numeric),
55             ("cat", Pipeline([
56                 ("imputer", SimpleImputer(strategy="most_frequent")),
57                 ("onehot", OneHotEncoder(handle_unknown="ignore", min_frequency=25, sparse_output=True)),
58             ]), categorical),
59         ])
60         model = LogisticRegression(
61             C=0.7, max_iter=500, solver="liblinear", random_state=SEED
62         )
63     elif model_name == "HistGradientBoosting":
64         pre = ColumnTransformer([
65             ("num", SimpleImputer(strategy="median"), numeric),
66             ("cat", Pipeline([
67                 ("imputer", SimpleImputer(strategy="most_frequent")),
68                 ("ordinal", OrdinalEncoder(handle_unknown="use_encoded_value", unknown_value=-1)),
69             ]), categorical),
70         ], sparse_threshold=0)
71         model = HistGradientBoostingClassifier(
72             learning_rate=0.06, max_iter=220, max_leaf_nodes=31, min_samples_leaf=30,
73             l2_regularization=1.0, early_stopping=True, validation_fraction=0.12,
74             random_state=SEED,
75         )
76     else:
77         raise ValueError(model_name)
78     return Pipeline([("preprocess", pre), ("model", model)])
79
80
```

```

81 def choose_f2_threshold(y_true: np.ndarray, probability: np.ndarray) -> tuple[float, float]:
82     thresholds = np.linspace(0.03, 0.50, 189)
83     scores = [fbeta_score(y_true, probability >= threshold, beta=2, zero_division=0) for threshold in thresholds]
84     best = int(np.argmax(scores))
85     return float(thresholds[best]), float(scores[best])
86
87
88 def metric_row(y_true: np.ndarray, probability: np.ndarray, threshold: float) -> dict:
89     prediction = (probability >= threshold).astype(int)
90     tn, fp, fn, tp = confusion_matrix(y_true, prediction, labels=[0, 1]).ravel()
91     return {
92         "roc_auc": roc_auc_score(y_true, probability),
93         "pr_auc": average_precision_score(y_true, probability),
94         "brier": brier_score_loss(y_true, probability),
95         "log_loss": log_loss(y_true, probability),
96         "threshold": threshold,
97         "accuracy": accuracy_score(y_true, prediction),
98         "balanced_accuracy": balanced_accuracy_score(y_true, prediction),
99         "precision": precision_score(y_true, prediction, zero_division=0),
100        "recall": recall_score(y_true, prediction, zero_division=0),
101        "specificity": tn / (tn + fp),
102        "f1": f1_score(y_true, prediction, zero_division=0),
103        "f2": fbeta_score(y_true, prediction, beta=2, zero_division=0),
104        "tn": int(tn), "fp": int(fp), "fn": int(fn), "tp": int(tp),
105    }

```

1.3 严格纵向历史特征

来源: optimization_research_pass.py:63-154

作用: 按患者和 encounter 顺序生成此前观测次数、前次数值特征、前次诊断/出院路径和变化量; 默认不使用既往再入院标签, target-history 仅作为敏感性上限。

```
1 def add_longitudinal_features(frame: pd.DataFrame, include_target_history: bool):
2     frame = frame.copy()
3     ordered = frame.sort_values(["patient_nbr", "encounter_id"]).index
4     patient_groups = frame.loc[ordered].groupby("patient_nbr", sort=False)
5
6     frame.loc[ordered, "observed_prior_encounters"] = patient_groups.cumcount().to_numpy()
7     frame["observed_prior_encounters"] = frame["observed_prior_encounters"].fillna(0).astype(float)
8     frame["log1p_observed_prior_encounters"] = np.log1p(frame["observed_prior_encounters"])
9     frame["is_first_observed_encounter"] = (frame["observed_prior_encounters"] == 0).astype(int)
10
11     prev_numeric = [
12         "time_in_hospital", "num_lab_procedures", "num_procedures", "num_medications",
13         "number_outpatient", "number_emergency", "number_inpatient", "number_diagnoses",
14         "prior_visits_total", "care_intensity",
15     ]
16     for column in prev_numeric:
17         previous = patient_groups[column].shift(1)
18         frame.loc[ordered, f"prev_{column}"] = previous.to_numpy()
19         frame[f"prev_{column}"] = frame[f"prev_{column}"].fillna(0)
20         frame[f"delta_{column}"] = frame[column] - frame[f"prev_{column}"]
21
22     previous_encounter = patient_groups["encounter_id"].shift(1)
23     frame.loc[ordered, "encounter_id_gap_from_previous"] = (
24         frame.loc[ordered, "encounter_id"] - previous_encounter
25     ).fillna(0).to_numpy()
26     frame["log1p_encounter_id_gap_from_previous"] = np.log1p(
27         frame["encounter_id_gap_from_previous"].clip(lower=0)
28     )
29
30     prev_categorical = [
31         "discharge_group", "admission_source_group", "admission_type_group",
32         "diag1_group", "diag2_group", "diag3_group", "insulin", "change", "A1Cresult",
33     ]
34     for column in prev_categorical:
35         previous = patient_groups[column].shift(1)
36         frame.loc[ordered, f"prev_{column}"] = previous.fillna("No previous").astype(str).to_numpy()
37
38     frame["same_prev_diag1_group"] = (
39         frame["diag1_group"].astype(str) == frame["prev_diag1_group"].astype(str)
40     ).astype(int)
41     frame["same_prev_discharge_group"] = (
42         frame["discharge_group"].astype(str) == frame["prev_discharge_group"].astype(str)
43     ).astype(int)
44     frame["current_prev_diag_path"] = (
45         frame["prev_diag1_group"].astype(str) + "|" + frame["diag1_group"].astype(str)
46     )
47     frame["current_prev_discharge_path"] = (
48         frame["prev_discharge_group"].astype(str) + "|" + frame["discharge_group"].astype(str)
49     )
50
51     numeric = [
52         "observed_prior_encounters", "log1p_observed_prior_encounters", "is_first_observed_encounter",
53         "encounter_id_gap_from_previous", "log1p_encounter_id_gap_from_previous",
54         "same_prev_diag1_group", "same_prev_discharge_group",
55     ]
56     for column in prev_numeric:
57         numeric.extend([f"prev_{column}", f"delta_{column}"])
58     categorical = [f"prev_{column}" for column in prev_categorical] + [
59         "current_prev_diag_path", "current_prev_discharge_path",
60     ]
61
62     if include_target_history:
63         status = frame.loc[ordered, "readmitted"].astype(str)
64         previous_status = patient_groups["readmitted"].shift(1)
65         frame.loc[ordered, "prev_readmitted_status"] = previous_status.fillna("No previous").astype(str).to_numpy()
66         is_30 = status.eq("<30").astype(int)
67         is_any = status.ne("NO").astype(int)
68         prior_30 = is_30.groupby(frame.loc[ordered, "patient_nbr"], sort=False).cumsum() - is_30
69         prior_any = is_any.groupby(frame.loc[ordered, "patient_nbr"], sort=False).cumsum() - is_any
70         prior_no = (
71             status.eq("NO").astype(int).groupby(frame.loc[ordered, "patient_nbr"], sort=False).cumsum()
72             - status.eq("<30").astype(int)
73         )
74         frame.loc[ordered, "prior_observed_30d_count"] = prior_30.to_numpy()
75         frame.loc[ordered, "prior_observed_any_readmit_count"] = prior_any.to_numpy()
76         frame.loc[ordered, "prior_observed_no_readmit_count"] = prior_no.to_numpy()
77         denom = frame.loc[ordered, "observed_prior_encounters"].replace(0, np.nan)
78         frame.loc[ordered, "prior_observed_30d_rate"] = (prior_30 / denom).fillna(0).to_numpy()
79         frame.loc[ordered, "prior_observed_any_readmit_rate"] = (prior_any / denom).fillna(0).to_numpy()
80         frame["previous_was_30d"] = frame["prev_readmitted_status"].eq("<30").astype(int)
```

```

81     frame["previous_was_any_readmit"] = frame["prev_readmitted_status"].isin(["<30", ">30"]).astype(int)
82     frame["prev_readmit_current_dx"] = (
83         frame["prev_readmitted_status"].astype(str) + "|" + frame["diag1_group"].astype(str)
84     )
85     numeric.extend([
86         "prior_observed_30d_count", "prior_observed_any_readmit_count",
87         "prior_observed_no_readmit_count", "prior_observed_30d_rate",
88         "prior_observed_any_readmit_rate", "previous_was_30d", "previous_was_any_readmit",
89     ])
90     categorical.extend(["prev_readmitted_status", "prev_readmit_current_dx"])
91
92     return frame, unique(numeric), unique(categorical)

```

1.4 历史模型训练与验证集融合

来源: optimization_history_refinement.py:43-170

作用: 训练多组 CatBoost 历史模型, 并只根据验证集 AP/PR-AUC 搜索融合权重, 测试集用于锁定后评价。

```
1 def cat_configs():
2     base_no_bagging = {k: v for k, v in BASE_CAT_PARAMS.items() if k != "bagging_temperature"}
3     return [
4         ("prefix_regularized_history", "ultra_prefix_no_raw_diag", {
5             **BASE_CAT_PARAMS, "iterations": 620, "depth": 5, "learning_rate": 0.038,
6             "l2_leaf_reg": 24.0, "random_strength": 0.45, "bagging_temperature": 0.75,
7             "random_seed": 3407,
8         }),
9         ("prefix_slow_history", "ultra_prefix_no_raw_diag", {
10             **BASE_CAT_PARAMS, "iterations": 520, "depth": 6, "learning_rate": 0.046,
11             "l2_leaf_reg": 9.0, "random_strength": 0.25, "bagging_temperature": 0.55,
12             "random_seed": 3407,
13         }),
14         ("full_mvs_history", "ultra_full", {
15             **base_no_bagging, "iterations": 520, "depth": 6, "learning_rate": 0.050,
16             "l2_leaf_reg": 11.0, "random_strength": 0.20, "bootstrap_type": "MVS",
17             "subsample": 0.88, "random_seed": 3407,
18         }),
19         ("full_bernoulli_history", "ultra_full", {
20             **base_no_bagging, "iterations": 520, "depth": 6, "learning_rate": 0.050,
21             "l2_leaf_reg": 10.5, "random_strength": 0.18, "bootstrap_type": "Bernoulli",
22             "subsample": 0.82, "random_seed": 3407,
23         }),
24     ]
25
26
27 def fit_history_models(frame, ultra_num, ultra_cat, history_num, history_cat, train_idx, val_idx, test_idx, label):
28     y_train = frame.loc[train_idx, "target_30d"].to_numpy()
29     y_val = frame.loc[val_idx, "target_30d"].to_numpy()
30     y_test = frame.loc[test_idx, "target_30d"].to_numpy()
31     rows = []
32     predictions = {}
33     feature_cache = {}
34     for config_name, variant, params in cat_configs():
35         if variant not in feature_cache:
36             base_num, base_cat = feature_variant(frame, ultra_num, ultra_cat, variant)
37             numeric = unique(base_num + history_num)
38             categorical = unique(base_cat + history_cat)
39             features = catboost_frame(frame, numeric, categorical)
40             cat_positions = [features.columns.get_loc(col) for col in categorical]
41             feature_cache[variant] = (features, cat_positions, len(numeric), len(categorical))
42         features, cat_positions, n_num, n_cat = feature_cache[variant]
43         name = f"{label}_{config_name}"
44         started = time.time()
45         model = CatBoostClassifier(**params)
46         model.fit(features.loc[train_idx], y_train, cat_features=cat_positions, verbose=False)
47         runtime = time.time() - started
48         val_probability = model.predict_proba(features.loc[val_idx])[:, 1]
49         test_probability = model.predict_proba(features.loc[test_idx])[:, 1]
50         predictions[name] = (val_probability, test_probability)
51         rows.append(score(
52             name, "validation", y_val, val_probability,
53             history_setting=label, variant=variant, config=config_name,
54             runtime_seconds=runtime, numeric_features=n_num, categorical_features=n_cat,
55         ))
56         rows.append(score(
57             name, "test", y_test, test_probability,
58             history_setting=label, variant=variant, config=config_name,
59             runtime_seconds=runtime, numeric_features=n_num, categorical_features=n_cat,
60         ))
61         print(f"{name}: val AP={rows[-2]['pr_auc']:.5f}, test AP={rows[-1]['pr_auc']:.5f}", flush=True)
62     return rows, predictions
63
64
65 def reconstruct_focused_extra():
66     ensemble = np.load(RES / "ensemble_probabilities.npz")
67     focused = np.load(RES / "focused_extra_probabilities.npz")
68     weights = {
69         "focused_prefix_regularized_depth5": 0.34,
70         "final_tree_ensemble": 0.21,
71         "focused_prefix_slow_depth6": 0.19,
72         "focused_full_mvs_depth6": 0.26,
73     }
74     parts = {
75         "focused_prefix_regularized_depth5": (
76             focused["val_focused_prefix_regularized_depth5"],
77             focused["test_focused_prefix_regularized_depth5"],
78         ),
79         "final_tree_ensemble": (ensemble["blend_val"], ensemble["blend_test"]),
80         "focused_prefix_slow_depth6": (
81             focused["val_focused_prefix_slow_depth6"],
```

```

82         focused["test_focused_prefix_slow_depth6"],
83     ),
84     "focused_full_mvs_depth6": (
85         focused["val_focused_full_mvs_depth6"],
86         focused["test_focused_full_mvs_depth6"],
87     ),
88 )
89 val = sum(weights[name] * parts[name][0] for name in weights)
90 test = sum(weights[name] * parts[name][1] for name in weights)
91 return val, test
92
93
94 def blend_grid(y_val, y_test, candidates, label):
95     validation_scores = {
96         name: average_precision_score(y_val, val)
97         for name, (val, _) in candidates.items()
98     }
99     top_names = sorted(validation_scores, key=validation_scores.get, reverse=True)[:5]
100     rows = []
101     step_count = 50
102
103     def compositions(total, size):
104         if size == 1:
105             yield [total]
106             return
107         for value in range(total + 1):
108             for rest in compositions(total - value, size - 1):
109                 yield [value] + rest
110
111     for size in [2, 3, 4]:
112         active = top_names[:size]
113         for units in compositions(step_count, size):
114             weights = [unit / step_count for unit in units]
115             val_probability = sum(weight * candidates[name][0] for name, weight in zip(active, weights))
116             test_probability = sum(weight * candidates[name][1] for name, weight in zip(active, weights))
117             rows.append({
118                 "blend_setting": label,
119                 "blend_size": size,
120                 "active_candidates": "+".join(active),
121                 "weights": json.dumps({name: float(weight) for name, weight in zip(active, weights)}),
122                 "validation_pr_auc": average_precision_score(y_val, val_probability),
123                 "validation_roc_auc": roc_auc_score(y_val, val_probability),
124                 "test_pr_auc": average_precision_score(y_test, test_probability),
125                 "test_roc_auc": roc_auc_score(y_test, test_probability),
126                 "test_brier": brier_score_loss(y_test, test_probability),
127             })
128     return pd.DataFrame(rows).sort_values("validation_pr_auc", ascending=False)

```

1.5 最终 strict-history 与 sensitivity 分支

来源: optimization_history_refinement.py:173-220

作用: 构造严格历史主模型和目标历史敏感性上限, 保证论文结论区分可部署模型与放宽预测时点后的上限。

```
1 def main():
2     started_all = time.time()
3     base_frame, _ = prepare_data()
4     ultra_frame, ultra_num, ultra_cat = add_ultra_features(base_frame)
5     train_idx, val_idx, test_idx, _ = patient_level_split(ultra_frame)
6     y_val = ultra_frame.loc[val_idx, "target_30d"].to_numpy()
7     y_test = ultra_frame.loc[test_idx, "target_30d"].to_numpy()
8
9     print("Fitting strict history models", flush=True)
10    strict_frame, strict_hist_num, strict_hist_cat = add_longitudinal_features(
11        ultra_frame, include_target_history=False
12    )
13    strict_rows, strict_predictions = fit_history_models(
14        strict_frame, ultra_num, ultra_cat, strict_hist_num, strict_hist_cat,
15        train_idx, val_idx, test_idx, "strict_history",
16    )
17
18    print("Fitting target-history sensitivity models", flush=True)
19    sensitivity_frame, sens_hist_num, sens_hist_cat = add_longitudinal_features(
20        ultra_frame, include_target_history=True
21    )
22    sensitivity_rows, sensitivity_predictions = fit_history_models(
23        sensitivity_frame, ultra_num, ultra_cat, sens_hist_num, sens_hist_cat,
24        train_idx, val_idx, test_idx, "target_history_sensitivity",
25    )
26
27    model_table = pd.DataFrame(strict_rows + sensitivity_rows).sort_values(
28        ["split", "pr_auc"], ascending=[True, False]
29    )
30    model_table.to_csv(TAB / "32_history_refinement_models.csv", index=False, encoding="utf-8-sig")
31
32    focused_extra_val, focused_extra_test = reconstruct_focused_extra()
33    research = np.load(RES / "research_hypothesis_probabilities.npz")
34    strict_candidates = {
35        "focused_extra_ensemble": (focused_extra_val, focused_extra_test),
36        **strict_predictions,
37    }
38    if "val_research_history_without_target_labels" in research.files:
39        strict_candidates["research_history_without_target_labels"] = (
40            research["val_research_history_without_target_labels"],
41            research["test_research_history_without_target_labels"],
42        )
43    sensitivity_candidates = {
44        **strict_candidates,
45        **sensitivity_predictions,
46    }
47    if "val_research_longitudinal_target_history_sensitivity" in research.files:
48        sensitivity_candidates["research_longitudinal_target_history_sensitivity"] = (
```

1.6 去 encounter-id gap 消融

来源: optimization_history_gap_ablation.py:41-135

作用: 移除由 encounter-id 差值构造的历史变量, 检验最终增益是否依赖伪时间间隔。

```
1  GAP_COLUMNS = {
2      "encounter_id_gap_from_previous",
3      "loglp_encounter_id_gap_from_previous",
4  }
5
6
7  def metric_row(model: str, y_val, p_val, y_test, p_test):
8      threshold, _ = choose_f2_threshold(y_val, p_val)
9      predicted = p_test >= threshold
10     return {
11         "model": model,
12         "roc_auc": roc_auc_score(y_test, p_test),
13         "pr_auc": average_precision_score(y_test, p_test),
14         "brier": brier_score_loss(y_test, p_test),
15         "log_loss": log_loss(y_test, np.clip(p_test, 1e-7, 1 - 1e-7)),
16         "threshold": threshold,
17         "accuracy": float((predicted == y_test).mean()),
18         "balanced_accuracy": balanced_accuracy_score(y_test, predicted),
19         "precision": precision_score(y_test, predicted, zero_division=0),
20         "recall": recall_score(y_test, predicted, zero_division=0),
21         "f2": fbeta_score(y_test, predicted, beta=2, zero_division=0),
22     }
23
24
25  def topk_rows(model: str, y, p):
26      rows = []
27      prevalence = float(y.mean())
28      order = np.argsort(-p)
29      n = len(y)
30      for fraction in [0.03, 0.05, 0.10, 0.20]:
31          k = max(1, int(np.ceil(n * fraction)))
32          chosen = order[:k]
33          true_positive = int(y[chosen].sum())
34          rows.append({
35              "model": model,
36              "capacity": fraction,
37              "selected_n": k,
38              "true_positive": true_positive,
39              "false_positive": int(k - true_positive),
40              "precision": true_positive / k,
41              "recall_capture": true_positive / int(y.sum()),
42              "lift": (true_positive / k) / prevalence,
43          })
44      return rows
45
46
47  def main():
48      started = time.time()
49      base_frame, _ = prepare_data()
50      ultra_frame, ultra_num, ultra_cat = add_ultra_features(base_frame)
51      train_idx, val_idx, test_idx, _ = patient_level_split(ultra_frame)
52      y_val = ultra_frame.loc[val_idx, "target_30d"].to_numpy()
53      y_test = ultra_frame.loc[test_idx, "target_30d"].to_numpy()
54
55      history_frame, history_num, history_cat = add_longitudinal_features(
56          ultra_frame, include_target_history=False
57      )
58      no_gap_num = [column for column in history_num if column not in GAP_COLUMNS]
59      removed = sorted(set(history_num) - set(no_gap_num))
60      print(f"Removed history numeric features: {removed}", flush=True)
61
62      model_rows, predictions = fit_history_models(
63          history_frame,
64          ultra_num,
65          ultra_cat,
66          no_gap_num,
67          history_cat,
68          train_idx,
69          val_idx,
70          test_idx,
71          "strict_history_no_gap",
72      )
73      pd.DataFrame(model_rows).to_csv(
74          TAB / "53_history_gap_ablation_models.csv",
75          index=False,
76          encoding="utf-8-sig",
77      )
78
79      focused_val, focused_test = reconstruct_focused_extra()
80      candidates = {"focused_extra_ensemble": (focused_val, focused_test), **predictions}
81      blends = blend_grid(y_val, y_test, candidates, "strict_history_no_gap")
```

```

82     blends.to_csv(TAB / "54_history_gap_ablation_blends.csv", index=False, encoding="utf-8-sig")
83     best = blends.iloc[0]
84     weights = json.loads(best["weights"])
85     no_gap_val = sum(weights[name] * candidates[name][0] for name in weights)
86     no_gap_test = sum(weights[name] * candidates[name][1] for name in weights)
87
88     full = np.load(RES / "history_final_probabilities.npz")
89     final_rows = [
90         metric_row("focused_extra_ensemble", y_val, full["focused_extra_val"], y_test, full["focused_extra_test"]),
91         metric_row(
92             "strict_history_ensemble_full",
93             y_val,
94             full["strict_history_ensemble_val"],
95             y_test,

```

1.7 五折患者级交叉验证

来源: optimization_history_cv.py:71-135

作用: 使用 StratifiedGroupKFold 进行患者互斥五折验证, 比较非历史模型与严格历史模型的跨划分稳定性。

```
1 def main():
2     started_all = time.time()
3     base_frame, _ = prepare_data()
4     ultra_frame, ultra_num, ultra_cat = add_ultra_features(base_frame)
5     strict_frame, history_num, history_cat = add_longitudinal_features(
6         ultra_frame, include_target_history=False
7     )
8     prefix_num, prefix_cat = feature_variant(ultra_frame, ultra_num, ultra_cat, "ultra_prefix_no_raw_diag")
9     history_prefix_num = unique(prefix_num + history_num)
10    history_prefix_cat = unique(prefix_cat + history_cat)
11
12    patient_target = strict_frame.groupby("patient_nbr")["target_30d"].max()
13    patient_labels = strict_frame["patient_nbr"].map(patient_target).to_numpy()
14    groups = strict_frame["patient_nbr"].to_numpy()
15    y = strict_frame["target_30d"].to_numpy()
16
17    splitter = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=SEED)
18    rows = []
19    for fold, (train_pos, valid_pos) in enumerate(splitter.split(strict_frame, patient_labels, groups), start=1):
20        y_valid = y[valid_pos]
21        print(f"Fold {fold}: train={len(train_pos)}, valid={len(valid_pos)}", flush=True)
22        p_no_history, seconds = fit_predict(ultra_frame, prefix_num, prefix_cat, train_pos, valid_pos)
23        row = score("prefix_catboost_no_history", fold, y_valid, p_no_history, seconds)
24        rows.append(row)
25        print(f"  no-history AP={row['pr_auc']:.5f}, ROC={row['roc_auc']:.5f}", flush=True)
26        pd.DataFrame(rows).to_csv(TAB / "42_history_group_cv.csv", index=False, encoding="utf-8-sig")
27
28        p_history, seconds = fit_predict(strict_frame, history_prefix_num, history_prefix_cat, train_pos, valid_pos)
29        row = score("prefix_catboost_strict_history", fold, y_valid, p_history, seconds)
30        rows.append(row)
31        print(f"  strict-history AP={row['pr_auc']:.5f}, ROC={row['roc_auc']:.5f}", flush=True)
32        pd.DataFrame(rows).to_csv(TAB / "42_history_group_cv.csv", index=False, encoding="utf-8-sig")
33
34    table = pd.DataFrame(rows)
35    summary_rows = []
36    for model, group in table.groupby("model"):
37        summary_rows.append({
38            "model": model,
39            "mean_pr_auc": group["pr_auc"].mean(),
40            "sd_pr_auc": group["pr_auc"].std(ddof=1),
41            "mean_roc_auc": group["roc_auc"].mean(),
42            "sd_roc_auc": group["roc_auc"].std(ddof=1),
43            "mean_brier": group["brier"].mean(),
44            "mean_log_loss": group["log_loss"].mean(),
45        })
46    summary = pd.DataFrame(summary_rows).sort_values("mean_pr_auc", ascending=False)
47    summary.to_csv(TAB / "43_history_group_cv_summary.csv", index=False, encoding="utf-8-sig")
48
49    pivot = table.pivot(index="fold", columns="model", values="pr_auc")
50    if {"prefix_catboost_strict_history", "prefix_catboost_no_history"}.issubset(pivot.columns):
51        deltas = pivot["prefix_catboost_strict_history"] - pivot["prefix_catboost_no_history"]
52    else:
53        deltas = pd.Series(dtype=float)
54    result = {
55        "runtime_seconds": time.time() - started_all,
56        "summary": summary.to_dict(orient="records"),
57        "fold_pr_auc_delta_history_minus_no_history": [float(x) for x in deltas.to_numpy()],
58        "mean_pr_auc_delta": float(deltas.mean()) if len(deltas) else None,
59        "all_folds_positive_delta": bool((deltas > 0).all()) if len(deltas) else None,
60    }
61    (RES / "history_group_cv_summary.json").write_text(
62        json.dumps(result, ensure_ascii=False, indent=2),
63        encoding="utf-8",
64    )
65    print(json.dumps(result, ensure_ascii=False, indent=2), flush=True)
```